02-session-system-log.md session-system-log[1]

ProjectEnhancer — Session Log / System Log

Document type: Operational event and audit specification
Status: v1.0
Purpose: Make every scan, decision, repair, enhancement, approval, and validation traceable without storing secrets or unnecessary source code.

1. Logging rules

- Logs are append-only and tied to a session, project, run, and base commit.
- Store metadata and evidence references, not raw secrets or full source files.
- Never log OAuth tokens, API keys, cookies, passwords, `.env` values, or private credentials.
- Every user-facing status must map to a system event.
- Errors must identify the phase, command category, exit result, and safe diagnostic summary.
- Logs distinguish **not available**, **not implemented**, **failed**, **skipped**, and **passed**.

2. Identifiers

```
session_id       User's active product session
project_id       Connected repository identity
run_id           One detection, repair, or enhancement run
base_commit      Immutable original commit SHA
branch           Isolated repair or enhancement branch
recommendation_id
approval_id
validation_id
```

3. Session lifecycle

```
SESSION_CREATED
  -> GITHUB_CONNECTED
  -> REPOSITORY_SELECTED
  -> DETECTION_STARTED
  -> STACK_CONFIRMATION_REQUIRED
  -> STACK_CONFIRMED
  -> COMPLETENESS_SCAN_STARTED
  -> SCOPE_SELECTED
  -> BASELINE_STARTED
  -> REPAIR_REVIEW or BASELINE_HEALTHY
  -> REPAIR_VALIDATION
  -> ENHANCEMENT_ANALYSIS
  -> RECOMMENDATION_REVIEW
  -> VERSION_2_VALIDATION
  -> PR_READY / RUN_PAUSED / RUN_CANCELLED
```

4. Required event fields

```
{
  "event_id": "evt_123",
  "event_type": "BASELINE_CHECK_COMPLETED",
```

```
  "timestamp": "2026-10-10T12:00:00Z",
  "session_id": "sess_123",
  "project_id": "proj_123",
  "run_id": "run_123",
  "base_commit": "abc123",
  "phase": "baseline",
  "status": "failed",
  "safe_summary": "Build failed with 2 TypeScript errors",
  "affected_paths": ["src/LoginForm.tsx"],
  "user_action_required": true
}
```

5. Important event types

Intake and detection

- `REPOSITORY_SELECTED`
- `SNAPSHOT_CREATED`
- `DETECTION_STARTED`
- `DETECTION_STEP_COMPLETED`
- `STACK_DETECTED`
- `STACK_CONFIRMATION_REQUIRED`
- `STACK_CONFIRMED`
- `STACK_RESCAN_REQUESTED`

Completeness and scope

- `COMPLETENESS_SCAN_STARTED`
- `COMPLETENESS_SCAN_COMPLETED`
- `PROJECT_MARKED_COMPLETE`
- `PROJECT_MARKED_PARTIAL`
- `SCOPE_MODE_SELECTED` with `FULL` or `EXISTING_ONLY`
- `AREA_EXCLUDED_AS_NOT_IMPLEMENTED`

Baseline and repair

- `BASELINE_STARTED`
- `CHECK_STARTED`
- `CHECK_COMPLETED`
- `BASELINE_FAILED`
- `REPAIR_PLAN_CREATED`
- `REPAIR_ITEM_APPROVED`
- `REPAIR_ITEM_REJECTED`
- `REPAIR_APPLIED`
- `REPAIR_VALIDATION_COMPLETED`

Enhancement and delivery

- `RECOMMENDATIONS_CREATED`
- `RECOMMENDATION_VIEWED`
- `RECOMMENDATION_APPROVED`
- `RECOMMENDATION_REJECTED`
- `VERSION_2_BRANCH_CREATED`

- CHANGE_MANIFEST_CREATED
- VERSION_2_VALIDATION_COMPLETED
- DIFF_READY
- PULL_REQUEST_CREATED

6. Example session timeline

```
09:00 REPOSITORY_SELECTED         owner/dashboard-app
09:00 SNAPSHOT_CREATED            base_commit=abc123
09:00 DETECTION_STARTED           evidence scan
09:00 STACK_DETECTED              React, Node.js, TypeScript
09:01 STACK_CONFIRMED             user approved
09:01 COMPLETENESS_SCAN_COMPLETED 3 partial areas, 1 placeholder
09:02 SCOPE_MODE_SELECTED         EXISTING_ONLY
09:02 BASELINE_STARTED            build, tests, security, runtime
09:04 BASELINE_FAILED             API health + 3 integration tests
09:05 REPAIR_PLAN_CREATED         2 repair items
09:07 REPAIR_ITEM_APPROVED        repair-001
09:10 REPAIR_VALIDATION_COMPLETED passed
09:11 ENHANCEMENT_ANALYSIS        existing areas only
09:13 RECOMMENDATIONS_CREATED     18 recommendations
```

7. Duck review record

Whenever the user says **“Duck”**, create a review event and compare the current product state against the four control documents:

```
DUCK_REVIEW_REQUESTED
DUCK_REVIEW_COMPLETED
```

The review must record: aligned items, off-track items, missing controls, unresolved decisions, evidence checked, and the next corrective action. “Duck” is a review trigger, not permission to change code.

8. Retention and privacy

Keep audit metadata long enough to support rollback and project history. Redact secrets before persistence. Keep source excerpts only when needed for the user’s review, with access controls and expiration. Never copy private memory or credentials into repository documentation.